Bilan de projet
SAÉ 5.01 “Concordia”

I. Le projet **2**
- 1.1 Objectif et périmètre 2
- 1.2 Parties prenantes 2
- 1.3 Technologies 3
- 1.4 Délais 3
- 1.5 L'équipe 4

II. L'organisation retenue **5**
- 2.1 Cycle de vie et enchaînement des étapes 5
- 2.2 Volumétrie du projet 6
- 2.3 Pilotage du projet 7
- 2.4 La dimension humaine 8

III. Le résultat produit **12**
- 3.1 Les succès 12
- 3.2 Les dérives 12

Conclusion: ce que nous retenons pour nos prochains projets **13**

Annexe **15**

I. Le projet

1.1 Objectif et périmètre

Objectif: adapter le jeu de société Concordia en version numérique, en respectant l'intégralité des règles du jeu original.

Dans le périmètre	Hors périmètre
Moteur de jeu (tours, fin de partie, décompte des points de victoire)	Jeu en réseau/multi-postes
9 cartes personnalité + carte Concordia, et leurs effets	Extensions du jeu physique
Plateau, provinces, villes, routes	Application mobile
Joueurs, colons, maisons, marchandises	Sauvegarde longue durée de parties
Persistance des données de référence (Oracle ORDB)	
Interface graphique (Tkinter), lancement en ligne de commande	
IA basique pour les cartes	

1.2 Parties prenantes

Partie prenante	Rôle	Attente principale
MOA: équipe pédagogique	Commanditaire, valide le cahier des charges et les livrables	Conformité aux règles, qualité de code, démonstration
MOE: équipe S5A-2 (7 personnes)	Conception, réalisation, tests, démo	Livrer dans les délais
Utilisateur final	Joueur de Concordia	Une partie jouable et fidèle aux règles

1.3 Technologies

Langage: Python 3.12 (gestion des dépendances uv / pyproject.toml)
Persistance: Oracle ORDB (oracledb), objets relationnels
Architecture: MVC + pattern Builder (GameBuilder) et Strategy (Strategy, décisions des joueurs IA)
Tests: pytest (28 fichiers, 118 tests ; 112 tournent sans base de données, 6 nécessitent une base Oracle réelle)
Qualité: typage systématique, conformité PEP8/lint, documentation générée par annotations
Outils de pilotage: Jira (backlog, sprint, burndown), GitLab (branches + merge requests), Google Drive (documents), messagerie/présentiel (communication)

1.4 Délais

Jalon	Date	Livrable
Initialisation	2 sept.	Cahier des charges compris, dépôt Git, équipe constituée
1er livrable: conception	Objectif personnel: 3 sept. Date réelle: 11 sept.	Use cases, diagrammes de séquence, diagramme de classes, choix des patterns
Point d'avancement intermédiaire	7 sept.	Compte rendu d'avancement présenté à la MOA
Rétrospective d'équipe (env. 50% du projet)	8 sept.	“Chapeau la rétro !” (6 chapeaux de Bono)
2e livrable: clôture	11 sept.	Code livré, démo, soutenance, bilan de management

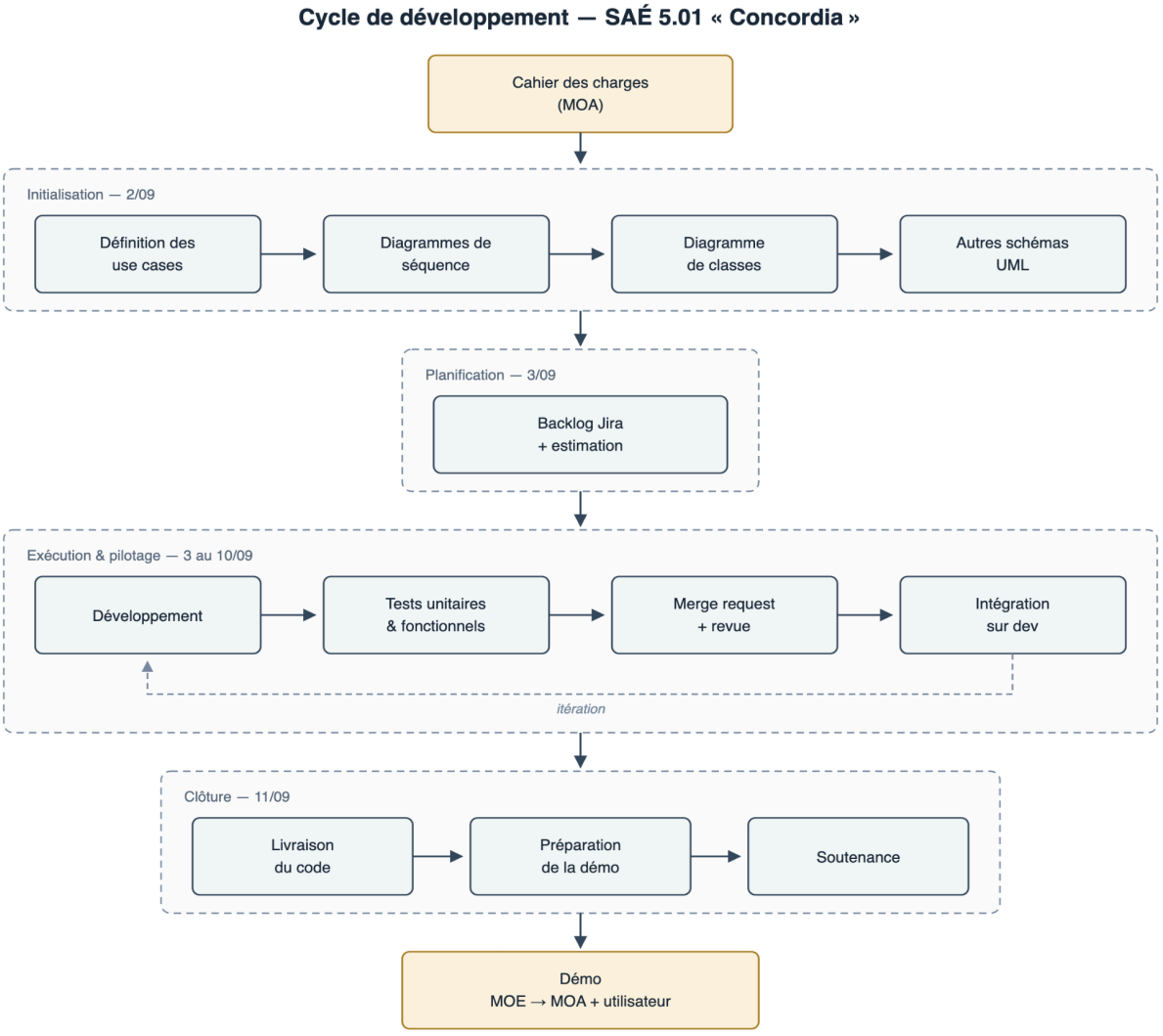
Contrainte forte: un seul cycle de 9 jours ouvrés, sans marge. Toute dérive se répercute directement sur la date de livraison, non négociable.

1.5 L'équipe

Membre	Fonction projet	Domaine principal
Talina Eugène-Combette	Scrum Master	Pilotage, backlog Jira, UML, animation des cérémonies, cartes personnalité
Evan Grosset-Janin	Tech Lead	Architecture, base de données, revue de code, intégration
Paco Laurent	Chef de projet	Moteur de jeu, calcul des points de victoire, cartes personnalité
Anaee Herbillon	Développeuse	Cartes personnalité, tests
Cédric Kuvan-Gee	Développeur	Structure de fichiers, cartes personnalité, UML
Mathis Gesson	Développeur	UML, cartes personnalité, IA
Noa Fessenmeyer	Développeur	UML, cartes personnalité, IA

II. L'organisation retenue

2.1 Cycle de vie et enchaînement des étapes



Deux étapes que nous avons voulu faire apparaître explicitement:

- La livraison du code est une étape distincte de la démonstration: merge de dev vers main, tag de version, README de lancement, jeu de données de référence chargé en base. « Le code est fini » et « le produit est livré » sont deux jalons différents.

- Le point d'avancement intermédiaire est préparé par la MOE seule: extraction du burndown Jira le 10/09 au soir, consolidation des points bloquants remontés par chaque développeur, rédaction du compte rendu par la Scrum Master, relecture par le Tech Lead avant présentation le 11/09.

2.2 Volumétrie du projet

Backlog Jira(cf. Figure 1)**: Sprint 0 (3 - 11 sept.)**

Indicateur	Valeur
Tickets créés	36 (+ 9 epics)
Points d'effort estimés	132
Points terminés au 8/09	51 (43%)
Points terminés au 11/09	127 (100%)
Composants/thèmes	9 (Game engine, Personality cards, GUI, Design/UML, Quality/Tests, Project management, Application architecture, Data/ORDB, Technical foundations)

Production logicielle

Indicateur	Valeur 8/09	Valeur 11/09
Commits	48	189
Contributeurs de code sur le dépôt	7 sur 7	7 sur 7
Branches de fonctionnalité	6 (initialization, calculateVP, Colonist, card-diplomat, card_architekt, card_tribune) + dev + main	13 encore actives (GUI, Game, ia, end_game, calculateVP, Colonist, card_architekt, card_consul, card_mercator, prefect_card, refactor_good, refactor_prefect, initialization) + dev + main
Fichiers Python	52	109
Lignes de code	~5 700	~14 900 (Écart dû à la documentations)

Charge humaine: 7 personnes × 8 jours ouvrés ≈ 49 jours-homme.
Coût matériel: postes de travail personnels et salle machine de l'IUT (pas de budget externe).

2.3 Pilotage du projet

Les objectifs/coûts/délais. Les coûts étant fixes (une salle, des postes, 7 personnes), le pilotage a porté exclusivement sur l'arbitrage périmètre/délai. Concrètement, le délai du 11/09 est intangible, donc c'est le périmètre qui a servi de variable d'ajustement.

Rituels et outils

Rituel/outil	Fréquence	But
Point debout en début de séance	Quotidien	Ce qui a avancé, ce qui bloque, qui fait quoi aujourd'hui
Backlog Jira	Continu	Priorisation, estimation en points, affectation, suivi d'état
Burndown chart	Continu	Détecter l'écart entre l'avancement réel et l'avancement théorique
Branche par fonctionnalité + merge request GitLab	Par tâche	Isoler le travail, forcer une relecture avant intégration sur dev
Compte rendu d'avancement	7/09	Rendre compte à la MOA à mi-parcours
Rétrospective “Chapeau la rétro !”	8/09	Faire remonter les irritants et décider des correctifs

Modes de management

Trois mécanismes complémentaires:

- Ajustement mutuel entre développeurs travaillant sur des cartes personnalité voisines (même classe mère, mêmes tests).
- Standardisation des procédures: typage obligatoire, conformité PEP8, une merge request par fonctionnalité, tests avant merge.
- Supervision directe limitée aux tâches critiques (moteur de tour, scoring, accès base de données), portées par le Tech Lead.
- Un chef de projet présent pour trancher dans les moments d’indécision.

Critères de répartition des tâches retenus: compétences déjà acquises, criticité de la tâche, appétences individuelles et disponibilité. À noter que l'acquisition d'expérience n'a pas été retenue comme critère.

2.4 La dimension humaine

Analyse Belbin[1]

Pôle Action (réalisation)	Pôle Réflexion (pensée critique, innovation)	Pôle Relation (communication, soutien)
Organisateur	Priseur	Promoteur
Propulseur	Concepteur	Soutien
Perfectionneur	Expert	Coordinateur

Relever les rôles sur les 7 feuilles ne suffit pas: une liste de rôles ne dit rien tant qu'on ne la met pas en regard du travail réellement produit. Le tableau ci-dessous fait ce lien: la colonne “tâches” est vérifiable dans Jira et dans l'historique Git. La colonne “rôles encadrés” vient de la grille remplie par la personne intéressée. La dernière colonne explicite le lien entre les deux.

Membre	Tâches réellement effectuées	Rôles encadrés	Lien tâches/rôle
Talina Eugène-Combette	Gestion de projet, animation des points quotidiens et de la rétrospective, diagrammes UML, carte Consul	Coordinateur	Seul membre à exercer une fonction de pilotage, donc pôle Relation.
Noa Fessenmeyer	Diagrammes UML, cartes Diplomat et Specialist	Promoteur, Soutien	Fait circuler l'information entre les binômes travaillant sur des cartes voisines
Mathis Gesson	Diagrammes de séquence et de classes, cartes Senator et Architect	Perfectionneur, Soutien	Sur la modélisation, il produit les livrables les plus normés du projet, ceux qui servent ensuite de référence commune à toute l'équipe.

Evan Grosset-Janin	Architecture MVC + GameBuilder, conception et écriture de la couche Oracle ORDB, revue de toutes les merge requests, intégration sur dev	Priseur, Expert	Il tranche les choix structurants, évaluation + spécialisation, donc pôle Réflexion.
Anaee Herbillon	Cartes Tribune et Mercator + tests associés	Perfectionneur, Soutien	Systématiquement “action + test” sur ses cartes, et relecture du code des autres membres travaillant sur le même socle de classes rigueur et disponibilité pour les collègues
Cédric Kuvan-Gee	Mise en place de l'arborescence du dépôt, carte Colonist, diagrammes UML	Organisateur, Soutien	Le travail de structuration des fichiers est du pur travail d'Organisateur
Paco Laurent	Moteur de calcul des points de victoire, branche calculateVP, fin de partie, carte Prefect	Propulseur	A pris seul l'initiative du chantier le plus complexe et le moins spécifié (le scoring). Énergie de démarrage caractéristique du Propulseur.

Répartition par pôle: le déséquilibre principal

Pôle Belbin	Membres dont c'est le pôle dominant	Lecture
Action (Organisateur, Propulseur, Perfectionneur)	2 / 7	Couverture faible pour un projet de production.
Réflexion (Priseur, Concepteur, Expert)	1 / 7	Le déséquilibre le plus critique: un seul membre se positionne en dominante Réflexion.
Relation (Promoteur, Soutien, Coordinateur)	4 / 7	Pôle largement dominant. Explique le très bon climat d'équipe et la difficulté à trancher les désaccords.

L'intérêt de l'équipe pour chaque rôle. En dépouillant les 7 grilles, nous avons compté le nombre de membres ayant marqué chaque rôle, qu'ils l'aient encadré ou entouré. L'indicateur obtenu n'est donc pas une photographie de ce que l'équipe fait, mais une mesure de l'intérêt collectif porté à chaque rôle donc ceux dans lesquels les membres se reconnaissent, ceux vers lesquels ils se projettent, et ceux qui ne suscitent ni l'un ni l'autre.

Rôle	Membres intéressés	Pôle
Soutien	6 / 7	Relation
Promoteur	5 / 7	Relation
Perfectionneur	5 / 7	Action
Concepteur	3 / 7	Réflexion
Coordinateur	3 / 7	Relation
Priseur	3 / 7	Réflexion
Propulseur	1 / 7	Action

- Le pôle Relation attire massivement (Soutien 6/7, Promoteur 5/7). L'équipe se reconnaît d'abord dans l'entraide et la circulation de l'information. C'est cohérent avec le très bon climat relevé au chapeau jaune de la rétrospective et c'est ce qui explique aussi la difficulté à trancher les désaccords.

- Le Perfectionneur à 5/7 est un signal positif sur un projet où la qualité de code était un critère de notation: le souci du détail est une valeur partagée, pas une contrainte subie. Cela se vérifie dans les faits, les tickets de typage et de conformité PEP8 ont été maintenus au backlog jusqu'au bout, alors qu'ils étaient les premiers candidats à l'abandon.
- Le Propulseur à 1/7 est le constat le plus lourd. Un seul membre joue ce rôle et aucun autre ne s'y projette. Ce n'est pas un déséquilibre passager qu'un réajustement de tâches suffirait à corriger. Une équipe sans énergie de relance dépend entièrement de son chef de projet pour créer l'élan.
- Le pôle Réflexion n'intéresse que 3/7 sur ses deux rôles principaux, alors que le projet reposait sur trois jours de modélisation, un choix d'architecture et un choix de patterns. L'équipe a donc laissé faire celui qui savait faire, sans que personne cherche à monter en compétence sur ces sujets.

Le principe retenu est le suivant:

- Chaque binôme mélange les pôles, pour ne pas reproduire à l'échelle du binôme le déséquilibre de l'équipe entière.
- Chaque membre est responsable d'un lot vertical (modèle + contrôleur + tests), et non d'un fichier.
- Un référent technique est désigné par binôme, ce qui décharge le Tech Lead de l'intégralité des arbitrages et compense la faiblesse du pôle Réflexion
- L'intérêt exprimé sur la grille de Belbin sert de critère d'affectation : lorsqu'une tâche pouvait être confiée à plusieurs personnes, elle est allée à celui ou celle qui avait marqué le rôle correspondant. C'est le seul moyen de faire évoluer la composition de l'équipe plutôt que de la subir.

Rétrospective “Chapeau la rétro!” (par Guillaume Ehret)

Nous avons choisi de faire une cérémonie scrum à environ 50% du projet. Le format de cette activité a pour but de passer une dizaine de minutes par chapeau, avec post-its individuels affichés au tableau et commentés devant l'équipe. (cf. figure 1, 2, 3, 4, 5, 6)

- Le chapeau noir converge sur un seul irritant de management: les changements soudains d'organisation. Ce n'est pas un problème technique, c'est un défaut de communication descendante. Une solution proposée est de faire une réunion avant un changement.

- Le format a joué son rôle de régulation: les tensions interpersonnelles ont été nommées publiquement, sur le ton de l'humour, sans que personne ne soit mis en cause. Ce cadre formel était un des moyens de faire remonter ces sujets.

- Le climat social comme critère d'efficacité. Au-delà de la performance (conformité du produit), l'équipe est restée viable: personne n'a décroché, et les décisions du chapeau bleu portent autant sur la cohésion (afterwork, communication) que sur la technique (explication du code).

III. Le résultat produit

3.1 Les succès

- Une architecture tenue de bout en bout. MVC respecté sur 52 fichiers, pattern Builder pour la construction de partie, couche d'accès Oracle isolée. Efficacité du code grâce aux UML fait en détail au début de projet.

- Une base de tests réelle, pas symbolique. 118 tests répartis sur 28 fichiers, tous au vert, couvrant le contrôleur de base de données, le game builder, les paramètres et les cartes, avec un marqueur integration isolant les tests nécessitant une base Oracle. Les tests ont été écrits en même temps que le code (“Card X (Action + Test)”), pas après.

- Une discipline Git respectée par les 7 membres. Une branche par fonctionnalité, merge request obligatoire, intégration sur dev avant main. 189 commits, aucun développement direct sur main.

- Une exigence de qualité tenue malgré la pression du délai. Typage systématique, conformité PEP8, documentation par annotations: ces tickets ont été maintenus au backlog alors qu'ils étaient les premiers candidats à l'abandon.

- Un climat d'équipe qui a tenu 9 jours sans conflit ouvert. Confirmé par la rétrospective: “bonne équipe”, “bonne ambiance de travail”, “participation de tous”. Aucun décrochage individuel.

- Une équipe qui s'auto-évalue en cours de route. L'analyse Belbin et la rétrospective ont été menées pendant le projet, pas après: elles ont débouché sur un réajustement effectif de la répartition des tâches.

3.2 Les dérives

Dérive 1: Changements soudains d'organisation
Constat n1 du chapeau noir, remonté par plusieurs membres indépendamment: “les changements soudains d'organisation”, “modifications imprévues”.

Causes: Les réaffectations de tâches décidées en réaction aux blocages étaient appliquées sans être annoncées ni expliquées. Effet secondaire du passage en mode directif : la vitesse de décision a été gagnée au détriment de l'adhésion.

Actions correctives: Décision n3 du chapeau bleu : toute réaffectation est annoncée à l'équipe avec sa justification, avant application. Systématisation du point debout comme canal unique d'annonce des changements.

Résultat: Ce problème ne s’est jamais reproduit par la suite ce qui nous amène à une meilleure fluidité de travail.

Dérive 2: Hétérogénéité des pratiques de code
Constat: “Différence dans les façons de coder” (chapeau noir). Constaté aussi dans l'historique Git: plusieurs commits de refactoring pur en fin de projet pour homogénéiser style, typage et messages.

Causes: Aucune convention écrite au démarrage, seulement “PEP8” comme référence implicite. Développement en 6 branches parallèles avec peu de lecture croisée du code des autres. Sur-représentation des profils “Soutien” : personne n'a voulu imposer un style à ses collègues.

Actions correctives: Ticket dédié à la conformité PEP/lint et au typage systématique. Décision n1 du chapeau bleu: explication régulière du code entre membres. Revue de merge request rendue obligatoire par le Tech Lead.

Résultat: Convergence obtenue, mais au prix d'un refactoring tardif, un coût qui aurait été évité par une convention écrite au jour 1.

Conclusion: ce que nous retenons pour nos prochains projets

Sur le management d'équipe

La composition d'une équipe se pilote, elle ne se subit pas. Sept profils techniques proches ont produit une équipe redondante sur le développement et fragile sur la coordination (un seul Coordinateur, un seul Propulseur). Faire l'analyse Belbin au démarrage plutôt qu'à mi-parcours aurait permis de répartir les tâches sur les comportements autant que sur les compétences.

Une équipe où tout le monde est “Soutien” ne remonte pas ses désaccords toute seule. Le bon climat n'est pas un indicateur d'absence de problème, c'est parfois ce qui les masque. Il faut un cadre formel (rétrospective, tour de table, canal anonyme) pour faire émerger ce que l'entraide spontanée étouffe. Les six chapeaux ont fait remonter en une heure des irritants vieux de plusieurs jours.

Changer d'organisation vite ne dispense pas d'expliquer. Notre principal irritant interne n'est pas venu des décisions elles-mêmes, mais de leur mode d'annonce. Passer en mode directif sous contrainte de délai est légitime. Le faire sans justifier fait perdre l'adhésion gagnée en mode participatif.

Piloter le moral autant que l'avancement. Aucun de nos indicateurs (burndown, tickets, commits) n'aurait détecté la fatigue exprimée au chapeau rouge. Un point de météo d'équipe hebdomadaire coûte cinq minutes.

Écrire les règles du jeu au jour 1. Conventions de code, définition de “terminé”, canal d'annonce des changements. Ce qui n'est pas écrit au démarrage se paie en refactoring et en frictions à la fin.

Sur la conduite de projet

Mesurer la valeur livrée, pas de l'activité. Un backlog découpé par composant technique indique combien de points ont été consommés. Il ne dit jamais ce qu'un utilisateur peut réellement faire avec le produit. C'est ce qui nous a empêchés de voir tôt que l'essentiel du travail se concentrerait sur la fin. Nous suivions notre consommation d'effort, pas notre progression fonctionnelle.

Un sprint de 9 jours n'est pas un sprint, c'est un projet en cascade déguisé. Sans point de recalage, la dérive ne se voit qu'à la fin. Trois sprints de 3 jours auraient donné deux occasions de corriger le tir.

Estimer sans vélocité de référence, c'est deviner. Notre estimation globale s'est révélée juste, mais le rythme ne l'était pas: 60 % des points livrés sur les trois derniers jours. Recalculer la vélocité après un premier sprint court l'aurait montré. C'est normal pour une première fois mais à condition de recalculer la vélocité après le premier sprint et de replanifier, ce qu'un sprint unique interdit.

La livraison est une étape à part entière, pas le point final du développement. Merge, tag, README, données de référence, démo: “le code est fini” et “le produit est livré” sont deux jalons distincts, et le second demande du temps qu'il faut avoir planifié.

Rendre compte, c'est analyser, pas décrire. Un compte rendu qui liste ce qui a été fait n'informe pas le commanditaire. Ce qui l'intéresse: où en est-on par rapport à l'objectif, quels sont les risques, qu'a-t-on décidé d'y faire. C'est vrai pour la MOA comme pour l'équipe.

Annexe

[1] : Analyse Belbin, mettre en regard les tâches réellement effectuées et les rôles endossés

Chacun a travaillé seul sur sa propre grille, puis nous avons dépouillé les 7 feuilles:

Rôle encadré = un rôle que le membre estime avoir déjà joué depuis le début du projet (2 à 3 maximum)
Rôle entouré = un rôle qu'il ou elle aimerait ou pense pouvoir jouer par la suite (2 à 3 maximum).

Un rôle marqué d'une manière ou de l'autre traduit donc un intérêt du membre pour ce rôle: soit il s'y reconnaît déjà, soit il s'y projette. C'est cet intérêt que nous avons dépouillé sur les 7 grilles, et il fonde le réajustement de notre répartition des tâches.

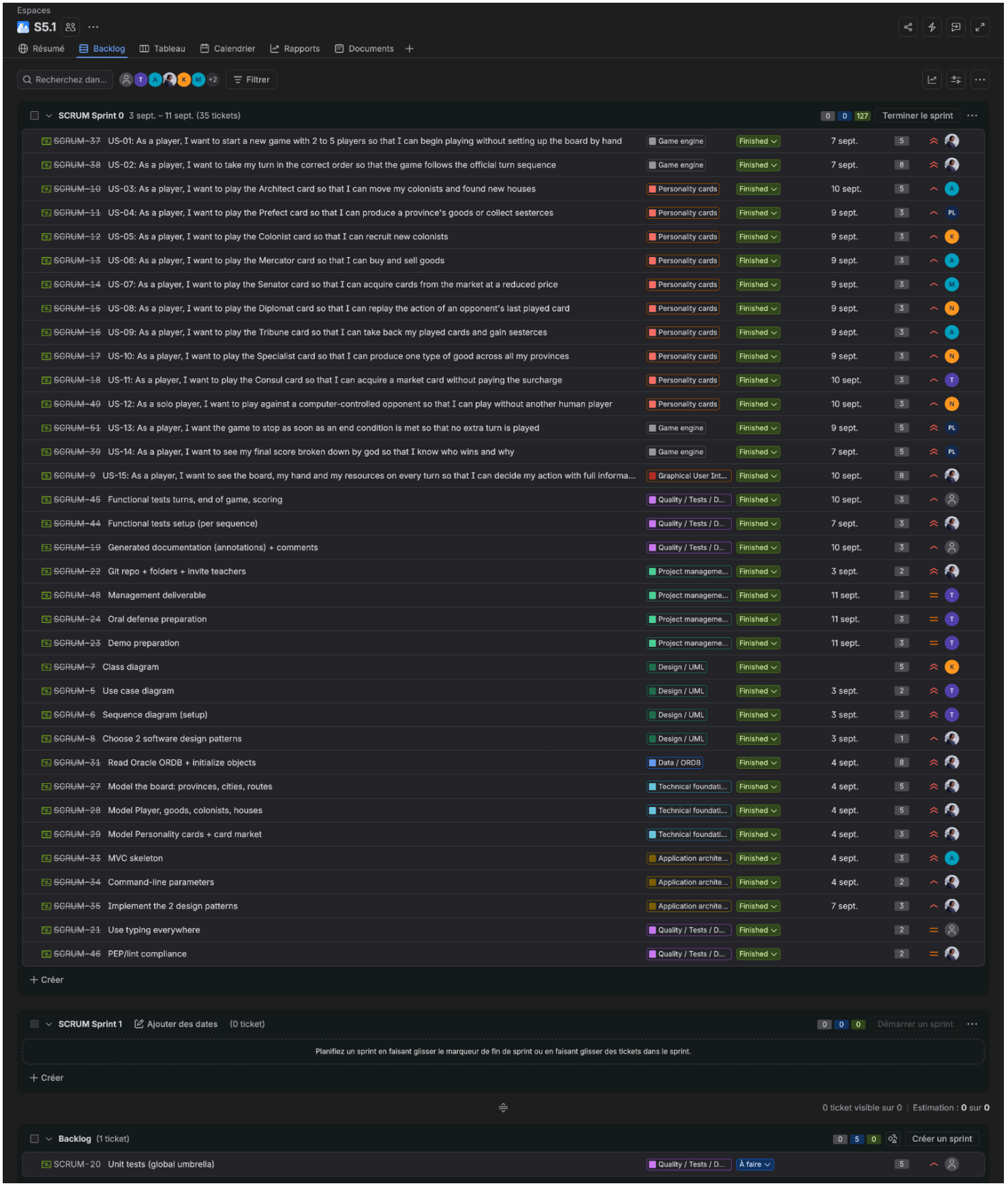

Espaces
S5.1
Résumé
Backlog
Tableau
Calendrier
Rapports
Documents
Recherchez dan...
Filtrer
SCRUM Sprint 0 3 sept. – 11 sept. (35 tickets)
Terminer le sprint
SCRUM-37 US-01: As a player, I want to start a new game with 2 to 5 players so that I can begin playing without setting up the board by hand
SCRUM-38 US-02: As a player, I want to take my turn in the correct order so that the game follows the official turn sequence
SCRUM-10 US-03: As a player, I want to play the Architect card so that I can move my colonists and found new houses
SCRUM-11 US-04: As a player, I want to play the Prefect card so that I can produce a province's goods or collect sesterces
SCRUM-12 US-05: As a player, I want to play the Colonist card so that I can recruit new colonists
SCRUM-13 US-06: As a player, I want to play the Mercator card so that I can buy and sell goods
SCRUM-14 US-07: As a player, I want to play the Senator card so that I can acquire cards from the market at a reduced price
SCRUM-15 US-08: As a player, I want to play the Diplomat card so that I can replay the action of an opponent's last played card
SCRUM-16 US-09: As a player, I want to play the Tribune card so that I can take back my played cards and gain sesterces
SCRUM-17 US-10: As a player, I want to play the Specialist card so that I can produce one type of good across all my provinces
SCRUM-18 US-11: As a player, I want to play the Consul card so that I can acquire a market card without paying the surcharge
SCRUM-40 US-12: As a solo player, I want to play against a computer-controlled opponent so that I can play without another human player
SCRUM-51 US-13: As a player, I want the game to stop as soon as an end condition is met so that no extra turn is played
SCRUM-39 US-14: As a player, I want to see my final score broken down by god so that I know who wins and why
SCRUM-9 US-15: As a player, I want to see the board, my hand and my resources on every turn so that I can decide my action with full informa...
SCRUM-45 Functional tests turns, end of game, scoring
SCRUM-44 Functional tests setup (per sequence)
SCRUM-19 Generated documentation (annotations) + comments
SCRUM-22 Git repo + folders + invite teachers
SCRUM-48 Management deliverable
SCRUM-24 Oral defense preparation
SCRUM-23 Demo preparation
SCRUM-7 Class diagram
SCRUM-5 Use case diagram
SCRUM-6 Sequence diagram (setup)
SCRUM-8 Choose 2 software design patterns
SCRUM-31 Read Oracle ORDB + initialize objects
SCRUM-27 Model the board: provinces, cities, routes
SCRUM-28 Model Player, goods, colonists, houses
SCRUM-29 Model Personality cards + card market
SCRUM-33 MVC skeleton
SCRUM-34 Command-line parameters
SCRUM-35 Implement the 2 design patterns
SCRUM-21 Use typing everywhere
SCRUM-46 PEP/lint compliance
Créer
SCRUM Sprint 1 Ajouter des dates (0 ticket)
Démarrer un sprint
Planifiez un sprint en faisant glisser le marqueur de fin de sprint ou en faisant glisser des tickets dans le sprint.
Créer
0 ticket visible sur 0 | Estimation : 0 sur 0
Backlog (1 ticket)
Créer un sprint
SCRUM-20 Unit tests (global umbrella)

Figure 1

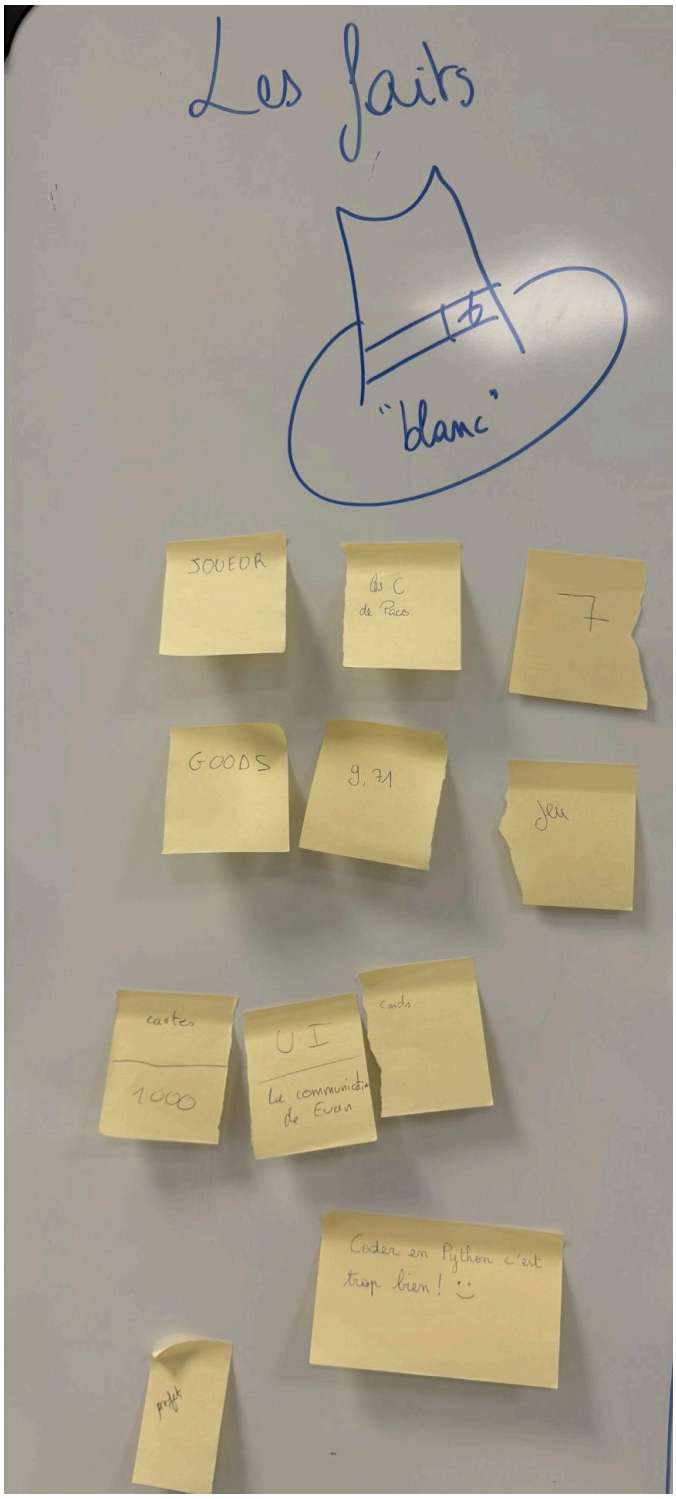


Figure 2

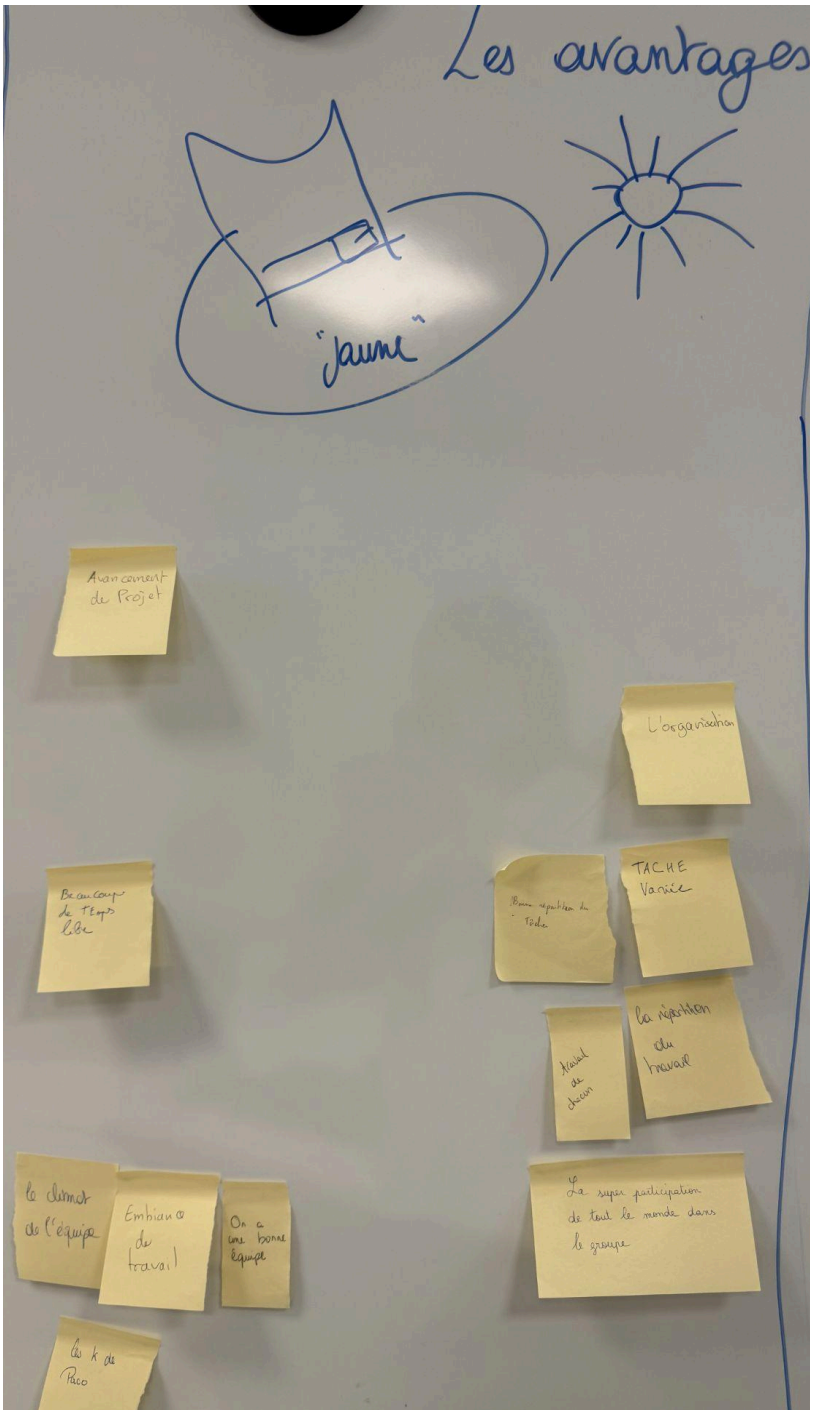


Figure 3

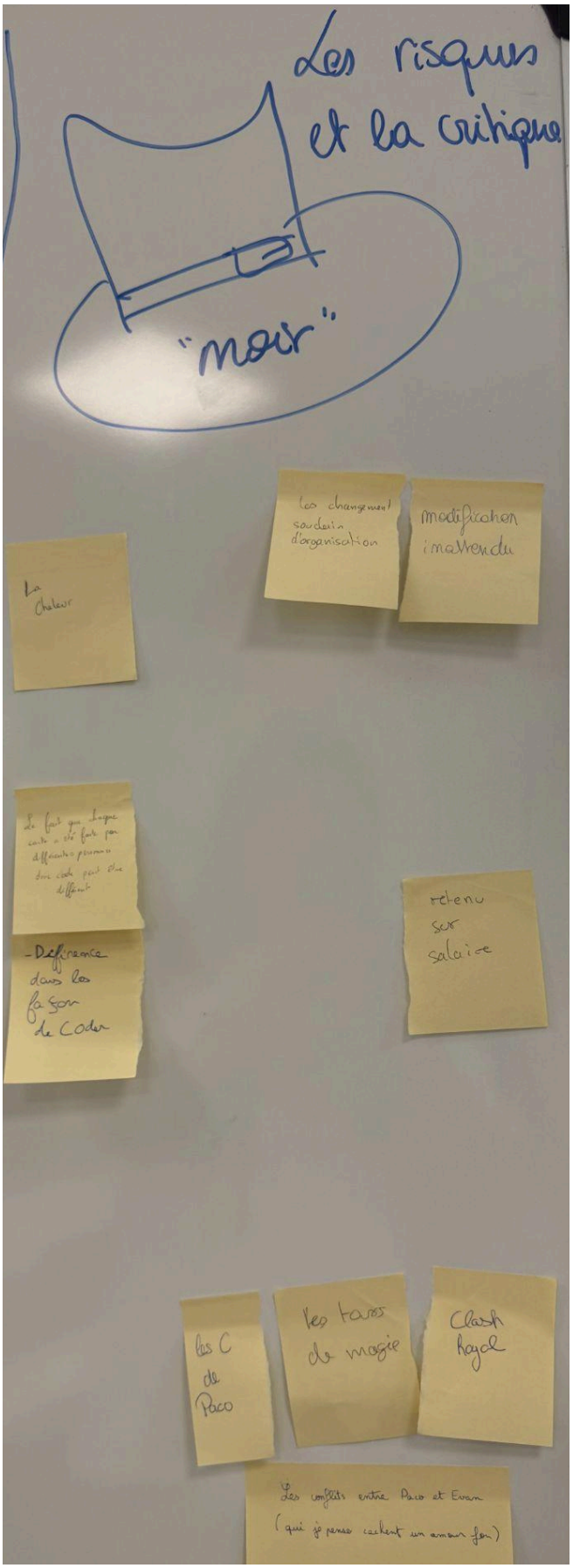


Figure 4

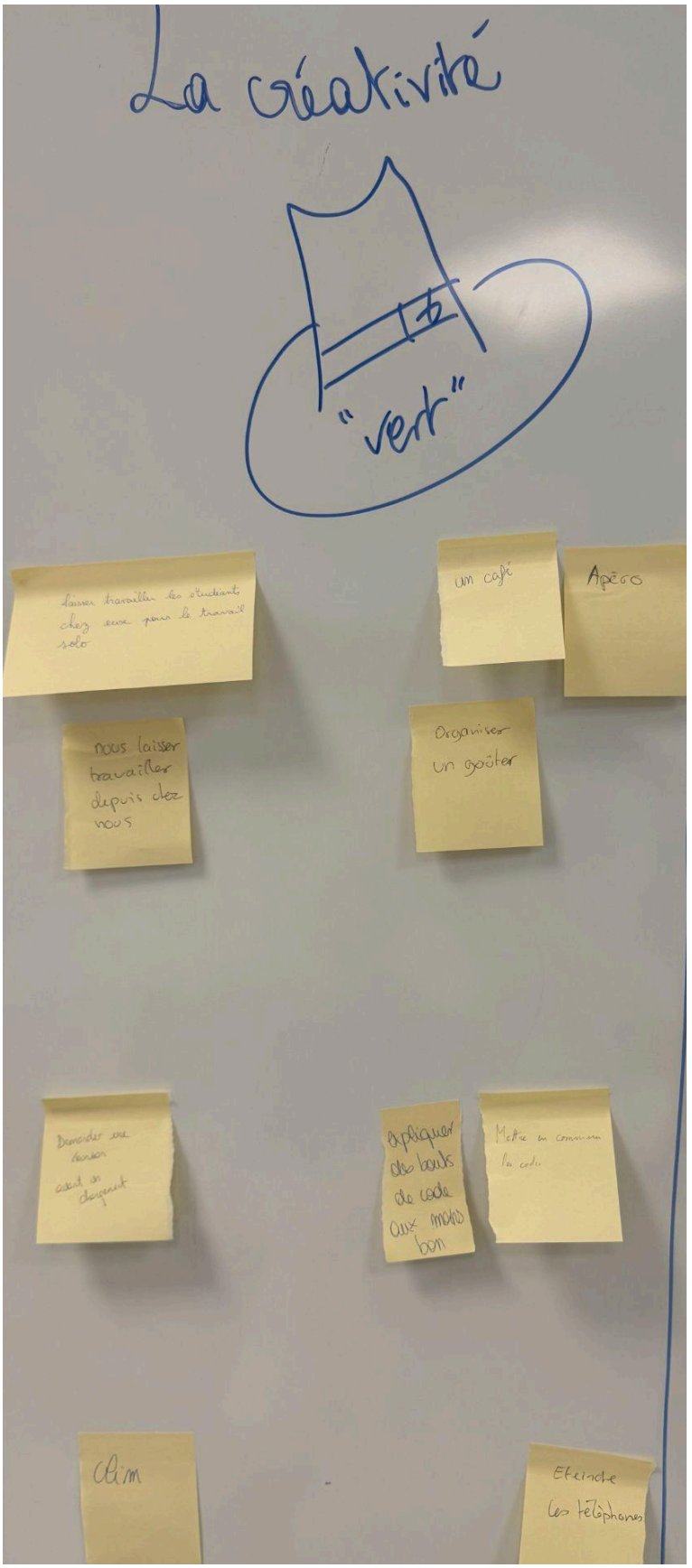


Figure 5

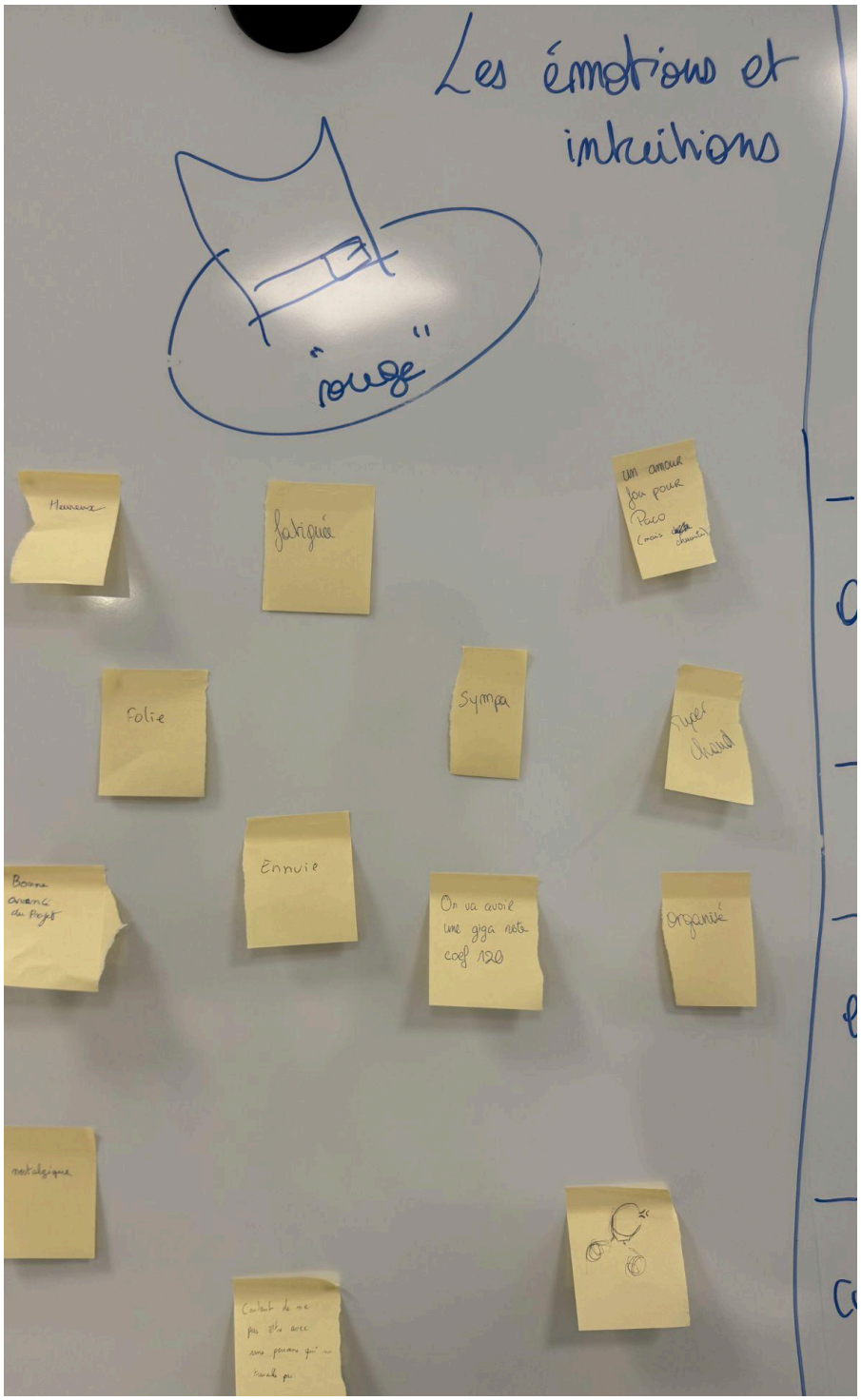


Figure 6

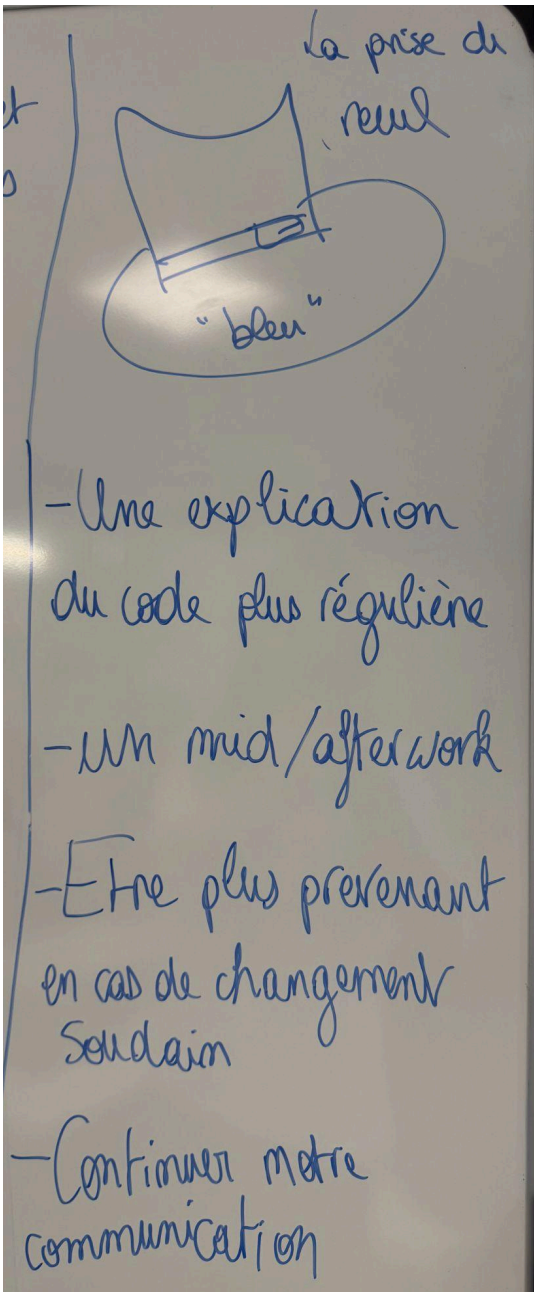


Figure 7